Blockchain and Decentralization

Lecture 2 — Data dissemination and mempool

Pierre Sutra

Télécom SudParis

CSC5006

version 1.0

Previously, in lecture 1

- blockchains try to *decentralize* IT services
- a (service) **transaction** is signed by its sender; miners group transactions in **blocks**, appended to a replicated **ledger**
- **hash chains** and **Merkle trees**: a short commitment to the past, and a logarithmic proof of membership
- **proof-of-work** turns computing power into a Sybil-resistant identity; **Nakamoto consensus**: the longest chain wins, eventually and with high probability
- Bitcoin assumes that less than half of the computing power is adversarial

Where we are

Lecture 1	blockchain 101, signatures, hash functions, Merkle trees, proof-of-work, Nakamoto consensus
Lecture 2	**safety and liveness, reliable broadcast, gossip, the mempool**
Lectures 3–5	agreement: consensus (FLP, Paxos), replicated state machines, Byzantine failures (PBFT)
Lectures 6–8	blockchains proper: distributed ledgers (Proof-of-X, layer 2), wallets, smart contracts and privacy, DeFi
Lectures 9–11	advanced topics: decentralized storage and compute, CRDTs, cross-chain transactions

Today

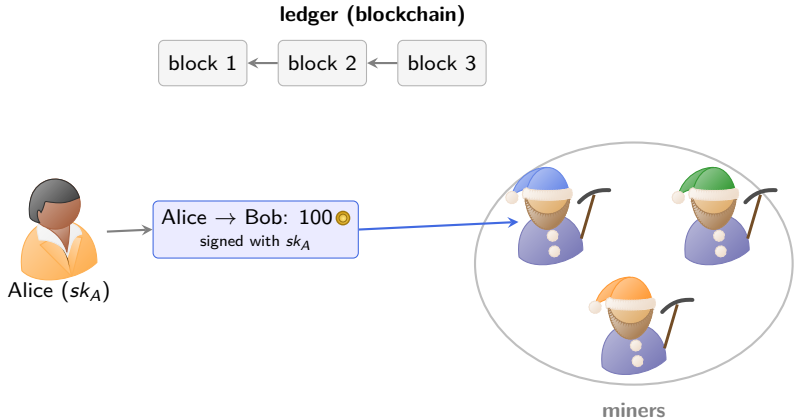
How to **disseminate** a message to a group of processes, with failures?

- the mempool: which transactions a node keeps, and when they are valid
- what we must specify, and how to reason about it (safety and liveness)
- reliable broadcast
- gossip protocols: anti-entropy and rumor mongering; how blockchain nodes gossip transactions (CometBFT)

Part	Time
Motivation	5’
Mempool	8’
Distributed computing	21’
Reliable broadcast	9’
Gossip	24’
Wrap-up	2’

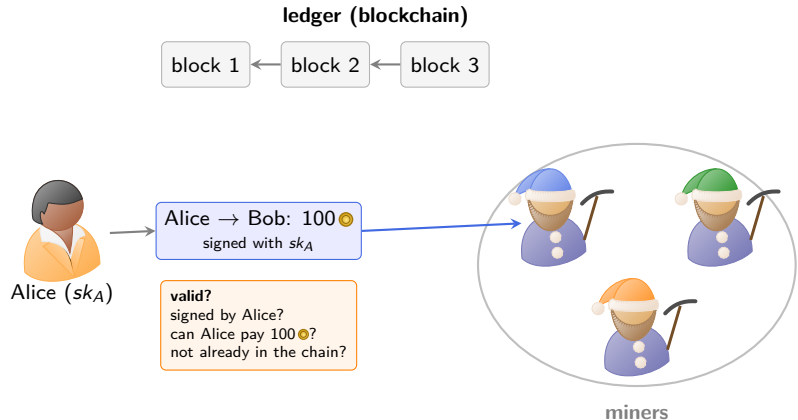


1. Motivation
2. The mempool: pending and valid transactions
3. Distributed computing: model and correctness
4. Reliable broadcast
5. Gossip-based protocols
6. Wrap-up

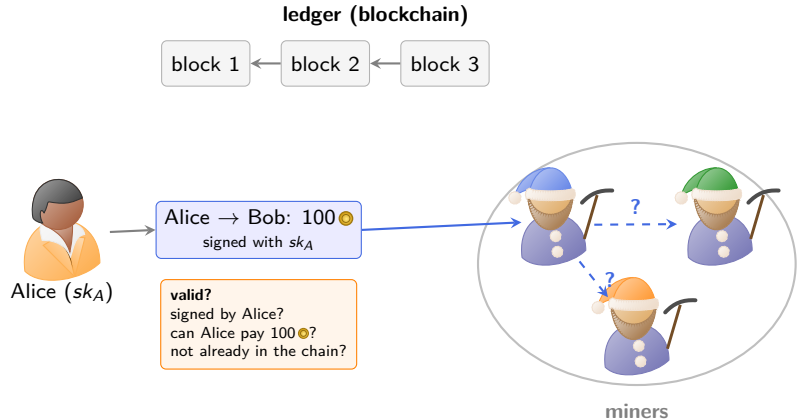


Alice signs a transaction, and sends it to **one** miner.

From a user to the blockchain

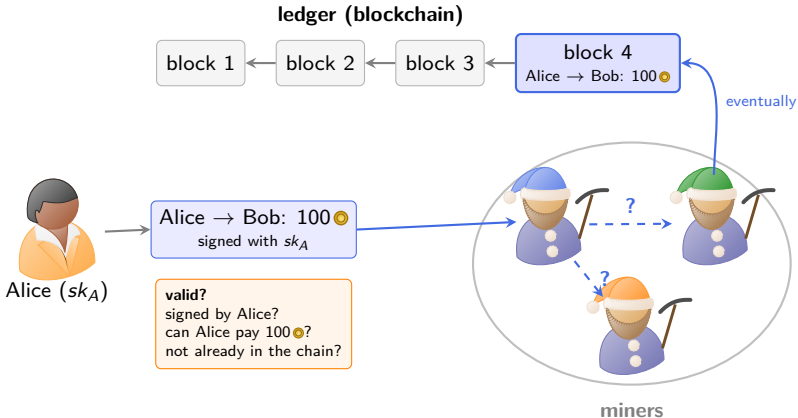


Question 1: when is the transaction *valid*, so that a miner keeps it, relays it, and puts it in a block?



Question 2: how does the transaction reach *every* miner, and in particular the one that mints the next block?

From a user to the blockchain



The goal: if Alice sends a **valid** transaction, it is **eventually** in the blockchain.

Goal

When a user sends a transaction, if the transaction is valid, then it is eventually added to the blockchain.

Goal

When a user sends a transaction, if the transaction is valid, then it is eventually added to the blockchain.

1. **When is a transaction valid?**
   - well signed, the sender can pay, and it is not a replay of an earlier transaction
   - validity depends on the state, which changes with every block

   → the mempool (section 2)

Goal

When a user sends a transaction, if the transaction is valid, then it is eventually added to the blockchain.

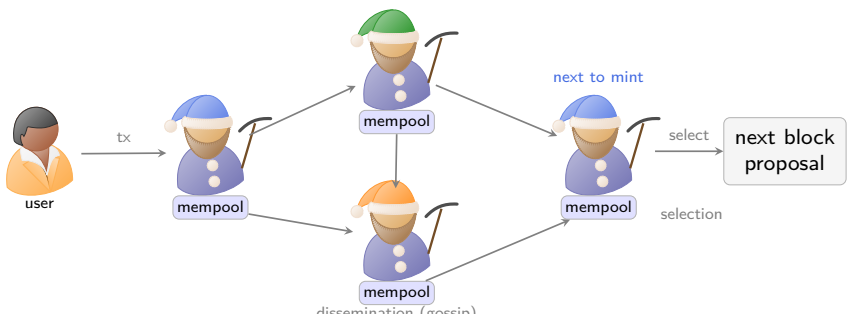
1. **When is a transaction valid?**
   - well signed, the sender can pay, and it is not a replay of an earlier transaction
   - validity depends on the state, which changes with every block

   → the mempool (section 2)
2. **How to disseminate a transaction?**
   - the user knows one node; the transaction must reach all of them, without a central server
   - nodes fail, join and leave, and none of them can be trusted to relay

   → distributed computing, reliable broadcast, gossip (sections 3–5)

Dissemination, then selection

- every miner keeps the pending transactions in its **mempool**, and relays them to its peers: the **network layer**
- the miner that mints the next block selects transactions from its own mempool (e.g., the highest fees)



1. Motivation
2. The mempool: pending and valid transactions
3. Distributed computing: model and correctness
4. Reliable broadcast
5. Gossip-based protocols
6. Wrap-up

- each node keeps the transactions it has heard of, and that are not yet in a block: its **memory pool** (*mempool*, *txpool*)
- it plays two roles

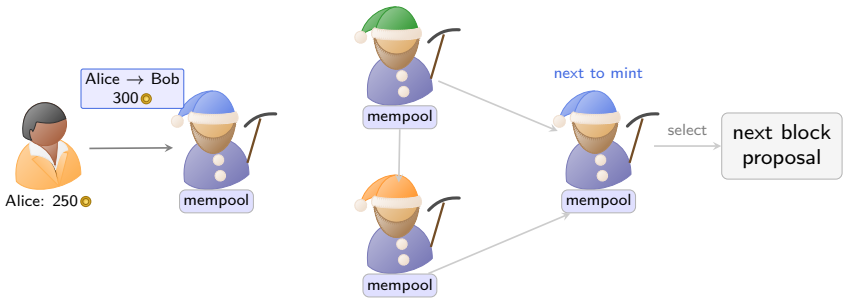
  Dissemination a transaction must reach every node quickly, so that it is visible as "pending" everywhere

  Selection a miner ranks pending transactions by fee, and fills the next block with the richest ones (Bitcoin, Ethereum); CometBFT takes them first come, first served
- a node keeps, and relays, only the transactions that are **valid** (see the next slide).

When is a transaction valid?

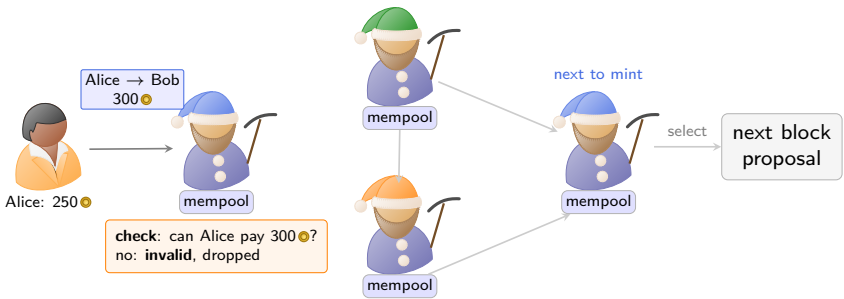
- validity is relative to the **current state**: the last block, plus the transactions accepted since
- Ethereum accepts a transaction when
  - it is well formed, and signed by its sender
  - its **nonce** is not below the nonce of the sender (next slide)
  - the sender can pay: value + *some fee* (lecture 7 details this)
- an invalid transaction is **dropped**, and **not relayed**: otherwise, flooding the network is free
- after each block, a node **rechecks** its pending transactions

An invalid transaction



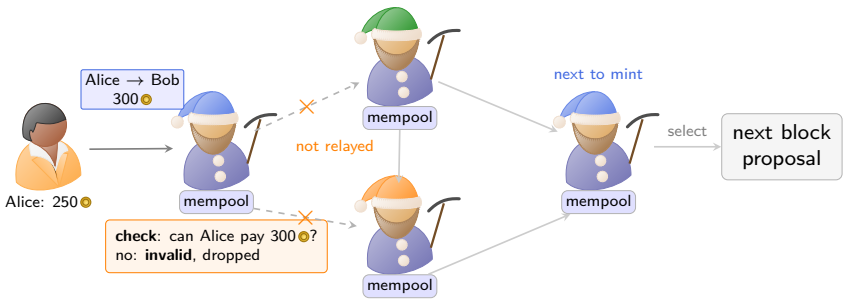
Alice owns 250. She signs a transaction that pays 300 to Bob, and sends it to a miner.

An invalid transaction



The miner checks the transaction against its current state: Alice cannot pay. The transaction is **invalid**: it is dropped, and never enters the mempool.

An invalid transaction



It is **not relayed** either: the other miners never see it, and it never reaches a block. Relaying it would let anyone flood the network for free.

- Alice signs “pay 10 to Bob”; the transaction is gossiped, and lands in a block
- Bob keeps its bytes, and broadcasts them again
- the signature is still Alice’s, and Alice can still pay: **is the transaction valid?**

Sending a transaction again?

- Alice signs “pay 10 to Bob”; the transaction is gossiped, and lands in a block
- Bob keeps its bytes, and broadcasts them again
- the signature is still Alice’s, and Alice can still pay: **is the transaction valid?**
- if it is, Bob replays it until Alice’s account is empty: a **replay attack**

Sending a transaction again?

- Alice signs "pay 10 to Bob"; the transaction is gossiped, and lands in a block
- Bob keeps its bytes, and broadcasts them again
- the signature is still Alice's, and Alice can still pay: **is the transaction valid?**
- if it is, Bob replays it until Alice's account is empty: a **replay attack**
- signature and balance are not enough: the state must also remember *which* transactions of Alice were executed

Sending a transaction again?

- Alice signs "pay 10 to Bob"; the transaction is gossiped, and lands in a block
- Bob keeps its bytes, and broadcasts them again
- the signature is still Alice's, and Alice can still pay: **is the transaction valid?**
- if it is, Bob replays it until Alice's account is empty: a **replay attack**
- signature and balance are not enough: the state must also remember *which* transactions of Alice were executed
- keeping all of them would be costly; a single **counter** per account suffices: the **nonce**

The nonce

- each account has a **nonce**: the number of transactions it has executed
- in a block, a transaction executes only if its nonce *equals* that of its sender, and increments it
- three roles

  Replay the same bytes, sent again, carry a stale nonce: executed at most once

  Order a sender's transactions execute in nonce order; a gap waits

  Replacement same nonce, higher fee: replaces a stuck transaction

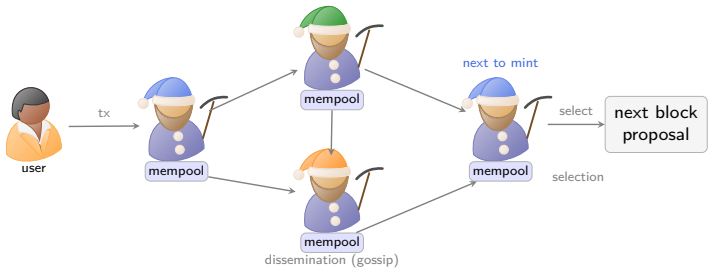
- Bitcoin: no nonce, instead an output is spent once (UTXO, lecture 6)

What a mempool must do

- **deduplicate**: a hash map from transaction ID to transaction
- **validate** before relaying: signature, nonce or inputs, balance, minimum fee; otherwise a node relays garbage for free (a denial-of-service vector)
- **recheck** after each block: drop what became invalid
- **order**: by arrival, a list (CometBFT); or by fee, a heap, for a miner to pop the best transactions (Ethereum)
- **bound its size**: when full, evict the lowest-fee transactions (Bitcoin Core: 300 MB by default)
- **replace**: let a user bump the fee of a stuck transaction (Ethereum: a replacement needs a fee bump of at least 10%, Bitcoin: BIP 125)

1. Motivation
2. The mempool: pending and valid transactions
3. Distributed computing: model and correctness
4. Reliable broadcast
5. Gossip-based protocols
6. Wrap-up

- so far, **one** node: what it keeps, and what it accepts (question 1)
- now, a valid transaction must reach **every** node, the next to mint in particular (question 2)
- no node sees the whole network; messages take time; nodes crash, join, and leave
- before a protocol: a **model** of the system, a **specification** of the problem



A distributed system is a software system in which components located on networked computers communicate and coordinate their actions by passing messages.

Definition

A distributed system consists of a set of processes that may either send a message, receive a message, or execute a local step.

Distributed computing is the field of computer science that studies distributed systems.

How to disseminate a message to a group of processes?

Question 2, in the words of distributed computing: a transaction is a message, and the nodes are processes. Well, we need to specify this ...

- is a broadcast or a unicast primitive available?
- are links reliable?
- can processes fail? What types of failures?
- must every non-faulty process receive the message?
- ...

To answer these questions, this lecture considers the **standard crash-failure model**.
In this model,

- the processes are *onymous*, *asynchronous*, and *crash-prone*,
- the communication graph is complete, and
- the links between processes are reliable.

anonymous / onymous a system is anonymous when processes have no identity (e.g., population protocols); otherwise it is onymous

synchronous / asynchronous in a synchronous system, there is a known upper bound on the time to execute a step (send, receive, local); otherwise it is asynchronous

crash / Byzantine what a faulty process can do

- *crash* (fail-stop): the process simply stops executing the program
- *Byzantine*: the process executes arbitrary code (lecture 4)

A process is **correct** when it does not fail.

Definition (reliable link)

The link from p to q is reliable when:

(integrity) q receives a message m at most once,

(validity) if q receives m, then m was sent by p, and

(termination) if p sends m to q, and both p and q are correct, then eventually q receives m.

1. **identities** help a lot and are relatively easy to create (e.g., a hash of a public key)

1. **identities** help a lot and are relatively easy to create (e.g., a hash of a public key)
2. **message losses** are common, but we may mask them (e.g., TCP)

Why is it the "standard" model?

1. **identities** help a lot and are relatively easy to create (e.g., a hash of a public key)
2. **message losses** are common, but we may mask them (e.g., TCP)
3. **asynchrony** is the common case: there is no upper bound on message delay

Why is it the "standard" model?

1. **identities** help a lot and are relatively easy to create (e.g., a hash of a public key)
2. **message losses** are common, but we may mask them (e.g., TCP)
3. **asynchrony** is the common case: there is no upper bound on message delay
4. **crashes** capture unexpected slowness (garbage collection, scheduling, memory contention, ...)

Why is it the "standard" model?

1. **identities** help a lot and are relatively easy to create (e.g., a hash of a public key)
2. **message losses** are common, but we may mask them (e.g., TCP)
3. **asynchrony** is the common case: there is no upper bound on message delay
4. **crashes** capture unexpected slowness (garbage collection, scheduling, memory contention, ...)

Byzantine behavior and permissionless identities are the subject of later lectures.

- is a broadcast or a unicast primitive available? **unicast**
- are links reliable? **yes**
- can processes fail? **yes, crashes**
- must every non-faulty process receive the message? ...

$\rightarrow$ how to specify this?

- building a system requires specifying its **properties**

[†] B. Alpern et al. "Recognizing Safety and Liveness". *Distributed Computing* (1987).

- building a system requires specifying its **properties**
- a property P defines a set of satisfying executions

```
int fact(int i) {
  return (i == 1) ?  1
    :  i * fact(i-1);
}
```

P_1: "variable i is always greater than 0"

P_2: "for every $i > 1$, if `fact(i)` is called, so is `fact(i-1)`"

[†] B. Alpern et al. "Recognizing Safety and Liveness". *Distributed Computing* (1987).

- building a system requires specifying its **properties**
- a property P defines a set of satisfying executions
- properties split into two categories: **safety** and **liveness**

```
int fact(int i) {
  return (i == 1) ?  1
    :  i * fact(i-1);
}
```

P_1: "variable i is always greater than 0"

P_2: "for every $i > 1$, if `fact(i)` is called, so is `fact(i-1)`"

[†] B. Alpern et al. "Recognizing Safety and Liveness". *Distributed Computing* (1987).

- building a system requires specifying its **properties**
- a property P defines a set of satisfying executions
- properties split into two categories: **safety** and **liveness**
- they are written as predicates over executions, usually with temporal logic (e.g., LTL); here we introduce them informally[†]

```
int fact(int i) {
  return (i == 1) ?  1
    :  i * fact(i-1);
}
```

P_1: "variable i is always greater than 0"

P_2: "for every $i > 1$, if `fact(i)` is called, so is `fact(i-1)`"

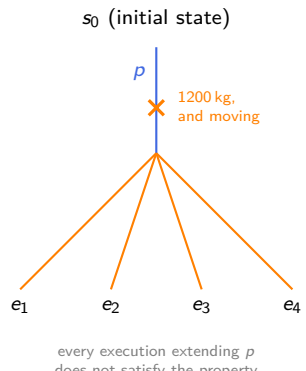
[†] B. Alpern et al. "Recognizing Safety and Liveness". *Distributed Computing* (1987).

Loosely speaking, **something bad never occurs**.

- "an elevator always weighs less than 1000 kg, or it is not moving"

Definition (safety)

P is a safety property iff, when an execution e does not satisfy P, there exists a prefix p of e such that every execution extending p does not satisfy P.



Loosely speaking, **something good eventually occurs**.

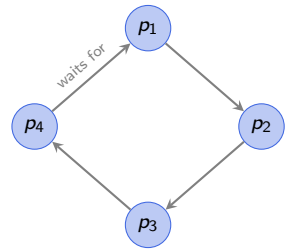
- "consider a traffic light T with state in $\{G, O, R\}$: it is always true that eventually T reaches a G state"

Definition (liveness)

P is a liveness property iff, when an execution e does not satisfy P, it is always possible to satisfy P by extending e.

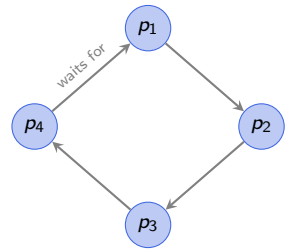
a safety violation is *final*; a liveness violation is never final

Liveness: progress conditions



a deadlock: a cycle of processes waiting for each other

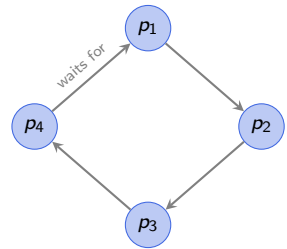
lock-freedom the system does not deadlock: some process always makes progress



a deadlock: a cycle of processes waiting for each other

lock-freedom the system does not deadlock: some process always makes progress

obstruction-freedom a process that takes steps alone always makes progress

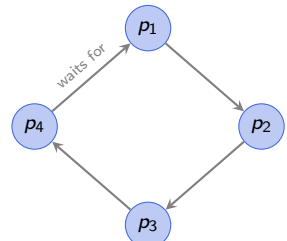


a deadlock: a cycle of processes waiting for each other

lock-freedom the system does not deadlock: some process always makes progress

obstruction-freedom a process that takes steps alone always makes progress

wait-freedom **every** process always makes progress



a deadlock: a cycle of processes waiting for each other

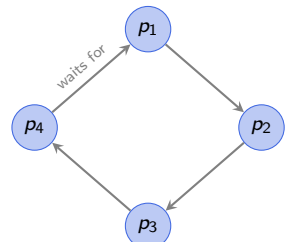
lock-freedom the system does not deadlock: some process always makes progress

obstruction-freedom a process that takes steps alone always makes progress

wait-freedom **every** process always makes progress

Wait-free $\Rightarrow$ lock-free $\Rightarrow$ obstruction-free.

- all objects in `java.util.concurrent` are lock-free; in practice, most are wait-free



a deadlock: a cycle of processes waiting for each other

Definition (reliable link)

(integrity) ? q receives a message m at most once,

(validity) ? if q receives m, then m was sent by p, and

(termination) ? if p sends m to q, and both are correct, then eventually q receives m.

For each property: **S** (safety) or **L** (liveness)?

Definition (reliable link)

(integrity) **S** q receives a message m at most once,

(validity) **S** if q receives m, then m was sent by p, and

(termination) **L** if p sends m to q, and both are correct, then eventually q receives m.

Integrity and validity are **safety** properties: a violation is visible in a finite prefix. Termination is a **liveness** property: the "eventually" can always be satisfied by extending the execution.

1. Motivation
2. The mempool: pending and valid transactions
3. Distributed computing: model and correctness
4. Reliable broadcast
5. Gossip-based protocols
6. Wrap-up

We fix a unique sender s and a message m. The API is `broadcast(m)` and `deliver(m)`.

(integrity)	a process delivers a message at most once
(validity)	if a process delivers m, then m was broadcast by s
(agreement)	if a correct process delivers m, then every correct process eventually delivers m
(termination)	if s is correct and broadcasts m, then eventually s delivers m

Setting: unicast only, reliable links, crash failures, complete communication graph.

We fix a unique sender s and a message m. The API is `broadcast(m)` and `deliver(m)`.

(integrity) ? a process delivers a message at most once

(validity) ? if a process delivers m, then m was broadcast by s

(agreement) ? if a correct process delivers m, then every correct process eventually delivers m

(termination) ? if s is correct and broadcasts m, then eventually s delivers m

Setting: unicast only, reliable links, crash failures, complete communication graph.

For each property: **S** (safety) or **L** (liveness)?

We fix a unique sender s and a message m. The API is `broadcast(m)` and `deliver(m)`.

(integrity) **S** a process delivers a message at most once

(validity) **S** if a process delivers m, then m was broadcast by s

(agreement) **L** if a correct process delivers m, then every correct process eventually delivers m

(termination) **L** if s is correct and broadcasts m, then eventually s delivers m

Setting: unicast only, reliable links, crash failures, complete communication graph.

A possible solution: flooding


```
1  received ← ∅
2  rbcast(m):
3    send m to all processes
4  upon receive(m):
5    if m ∉ received:
6      received ← received ∪ {m}
7      if self ≠ sender(m):
8        send m to all processes
9      deliver(m)
```

- the sender sends m to everybody
- upon the first reception, **relay** m to everybody, then deliver it
- the `received` set filters duplicates

The communication graph is complete.

Integrity the `received` set: m is delivered at most once

Validity links are reliable, so a received message was sent; only s creates messages

Termination if s is correct, it sends m to itself and delivers it

Agreement assume a process p delivers m. Either $p = s$, and s sent m to everybody; or p relayed m to everybody before delivering it. In both cases, since links are reliable, every correct process receives and delivers m

The relay before the delivery matters: the sender may crash right after sending m to a single process.

- each process relays m once: about n^2 messages for n processes
- the sender and every process need the identity of all the others: a complete graph
- fine for tens of processes, as in permissioned blockchains (lectures 3–5)

At large scale

Thousands to millions of nodes, no global knowledge, constant churn: a node knows only a small subset of the system. Flooding to everybody is not an option.

$\Rightarrow$ **gossip-based protocols**

1 Motivation

2 The mempool: pending and valid transactions

3 Distributed computing: model and correctness

4 Reliable broadcast

5 Gossip-based protocols

6 Wrap-up

Dynamicity

- nodes leave and join all the time
- failures are the common case, not the exception

Large size (thousands to millions of nodes)

- no global knowledge of the system
- each node knows only a small subset: its **view**

Every non-faulty process must receive the message **with high probability**.

Approach

- pair-wise exchanges of information between peers
- repeated, periodic interactions, with a **random** choice of partner
- eventually, all peers have the message w.h.p.

Properties

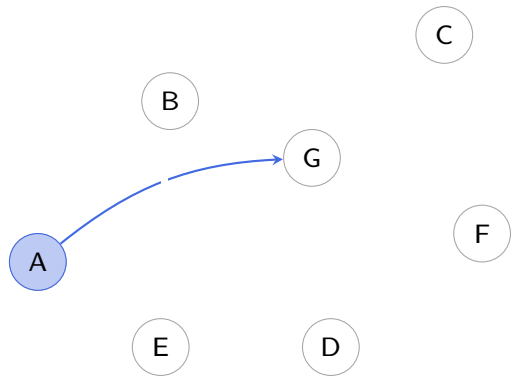
- robustness through randomness and diversity
- self-organization: the system evolves towards a correct state whatever its starting state
- the correction of faults is intrinsic

Epidemic vocabulary: a process is **infected** if it has the message, *susceptible* otherwise.[†]

[†] A. Demers et al. "Epidemic Algorithms for Replicated Database Maintenance". *PODC*. 1987.

Anti-entropy: an example

- A selects G at random to pass the message

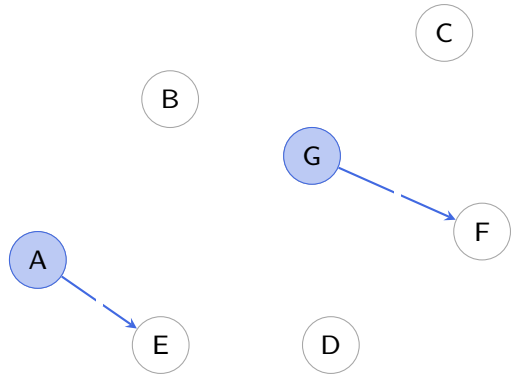


filled: infected; white: susceptible

Anti-entropy: an example

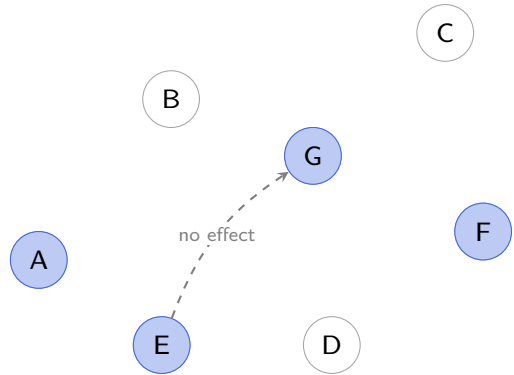
- A selects G at random to pass the message
- A selects E, G selects F

filled: infected; white: susceptible



Anti-entropy: an example

- A selects G at random to pass the message
- A selects E, G selects F
- E selects G: **no effect**

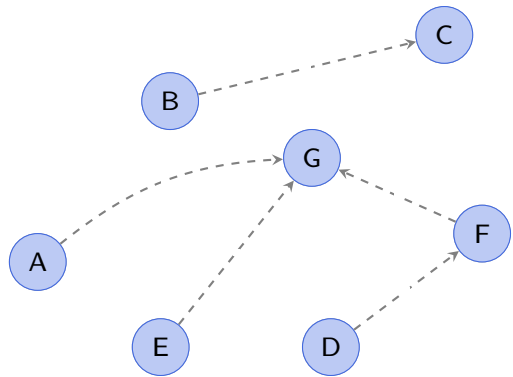


filled: infected; white: susceptible

Anti-entropy: an example

- A selects G at random to pass the message
- A selects E, G selects F
- E selects G: **no effect**
- eventually, all are infected, but exchanges go on: constant traffic

filled: infected; white: susceptible



Anti-entropy: when to synchronize?

Push if there is a new message, tell the other processes. Good when most processes are susceptible

Pull ask the other processes whether a message has to be delivered. Good when most processes are infected

Push-pull do both

+ every message is eventually delivered, even if local views are incomplete
− the dissemination delay is high: slow initial propagation, depending on the gossip frequency
− eventually, many useless requests

Theorem (Pittel 1987; Doerr and Kostrygin 2017), on a complete graph

When k of the n nodes are informed, an uninformed node becomes informed with probability

- k/n (pull)
- $1 - (1 - 1/n)^k$ (push)

and the expected spreading time of a message is

- $\mathbb{E}[T] = \log_2 n + \log_2 \ln n \pm O(1)$ (pull)
- $\mathbb{E}[T] = \log_2 n + \ln n \pm O(1)$ (push)

Logarithmic in the size of the system: for $n = 10^6$, about 24 rounds for pull and 34 for push, up to a constant.[†‡]

[†] B. Pittel. "On Spreading a Rumor". *SIAM Journal on Applied Mathematics* (1987).

[‡] B. Doerr et al. "Randomized Rumor Spreading Revisited". *ICALP*. 2017.

When receiving a message, propagate it to f other processes (f is the **fanout**).

- these f processes then propagate the message further
- exponential growth of the number of infected processes
- not a periodic protocol (but it can be modeled as one); also called (controlled) *flooding*
- similar to the spread of a rumor in a human population

Requires a stop condition

When must a node stop propagating a message? With no stop condition, the number of messages grows exponentially.

Rumor mongering: stop conditions (1/3)

Stateless conditions

- *probability-based*: stop with probability $1/k$
- *feedback-based*: stop with probability $1/k$ if the destination is already infected

\+ simple and stateless

− setting k appropriately requires knowing the size of the network

Rumor mongering: stop conditions (2/3)

Conditions based on the node's state

- maintain a memory of the messages delivered to the application
- propagate a message only when it is received for the first time

\+ simple

– the state can grow indefinitely: should we keep every message identifier?

– the effectiveness depends heavily on the (randomly chosen) first peers

Rumor mongering: stop conditions (3/3)

Conditions based on the message's state: hops-to-live (HTL)

- messages start with a counter; each propagation decreases it; stop when $HTL = 0$

\+ helps limit the number of messages

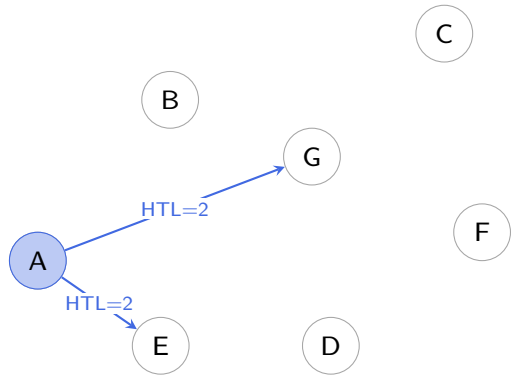
− the fraction of the network reached depends on the initial HTL

− hard to choose without knowing the number of nodes

In practice, a **mix** of these techniques.

Rumor mongering with hops-to-live: an example

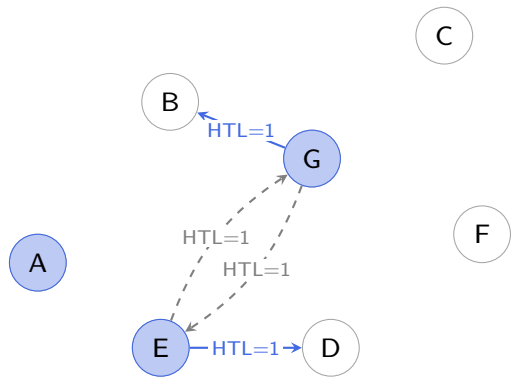
- A starts with $f = 2$ peers, $HTL = 2$



dashed: useless communications

Rumor mongering with hops-to-live: an example

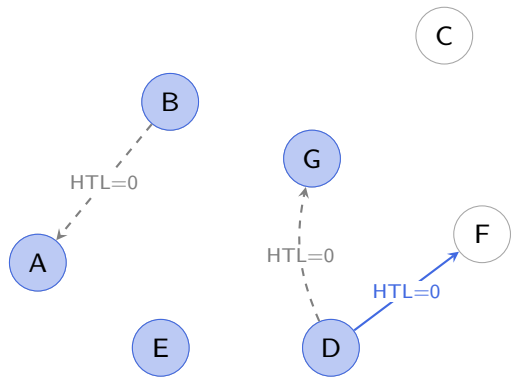
- A starts with $f = 2$ peers, $HTL = 2$
- E and G propagate with $HTL = 1$



dashed: useless communications

Rumor mongering with hops-to-live: an example

- A starts with $f = 2$ peers, $HTL = 2$
- E and G propagate with $HTL = 1$
- then with $HTL = 0$: a single message is useful

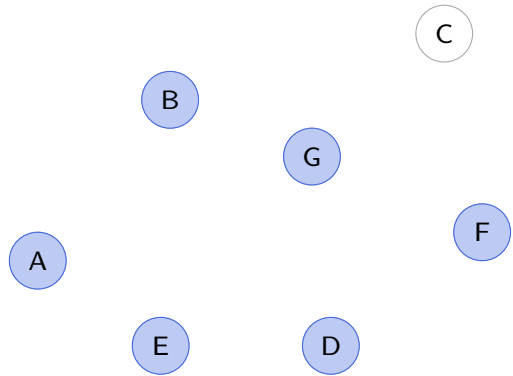


dashed: useless communications

Rumor mongering with hops-to-live: an example

- A starts with $f = 2$ peers, $HTL = 2$
- E and G propagate with $HTL = 1$
- then with $HTL = 0$: a single message is useful
- the rumor has stopped, but C **was not notified**

dashed: useless communications



Rumor mongering: what is the cost of a full coverage?

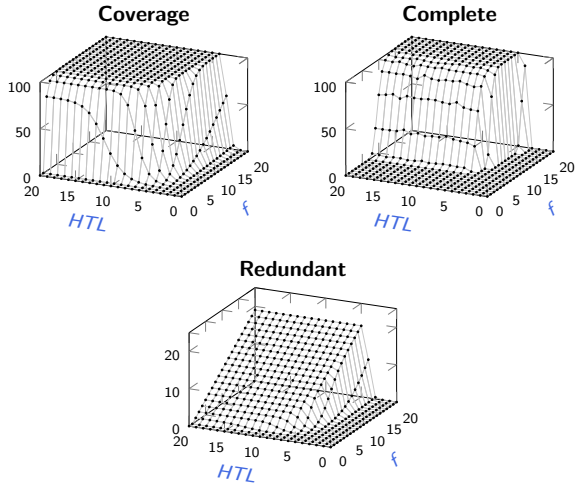
Dissemination of 1,000 messages to 10,000 nodes

Stop condition: Hops-to-Live

Coverage
% nodes receiving the message

Complete
% disseminations reaching all nodes

Redundant
ratio (# sent to infected) / (# sent to susceptibles) = *overhead*



Anti-entropy

+ ensures that all nodes get the update
− slow and expensive: constant traffic, even with no updates

Rumor mongering

− no guarantee that 100% of the nodes receive the update
+ fast propagation initially
− then many useless propagations to notified nodes

The best of both worlds

Rumor mongering first

- efficient at seeding the network
- stops before too many duplicates
- the stop condition can adapt to self-observation

Anti-entropy to complete

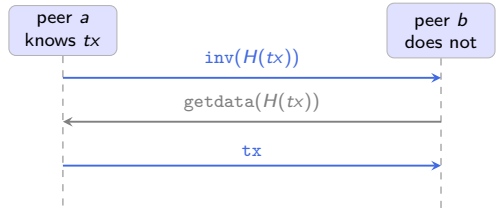
- more sources after the rumor phase: a more efficient anti-entropy
- reliability even if membership lists are not up to date

- what is gossiped: **transactions** (the mempool, section 2), **blocks**, and consensus **votes** (lecture 3)
- what makes it harder than in a data center
  - open membership: anyone may join, and some peers are *Byzantine* (lecture 4)
  - latency matters: a block that spreads late loses the race (lecture 1), a vote that arrives late is not counted
  - bandwidth matters: every node relays every transaction and every block
- two families
  - a stack of its own: Bitcoin, Ethereum's execution layer (devp2p), Solana, CometBFT
  - a shared library, **libp2p**: Ethereum's consensus layer, Filecoin, Polkadot, ...

Beyond blockchains, the cloud runs on gossip too: Apache Cassandra spreads cluster membership by gossip, and repairs its replicas by anti-entropy over Merkle trees (lecture 1).

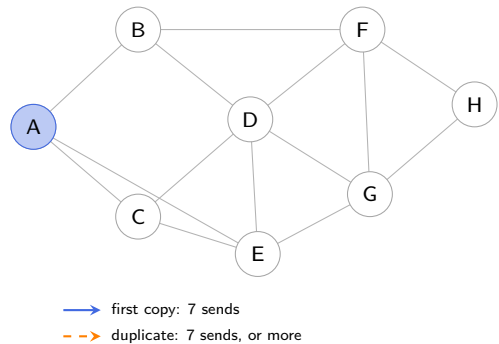
Gossip in practice: announce, request, send

- nodes keep a small number of **random** peers (Bitcoin: about 8 outgoing connections by default): the node's view
- sending the whole transaction to everybody wastes bandwidth: peers mostly know it already
- so a node first *announces* the hash of a transaction, and sends the body only to peers that ask
- this is a *push-pull* exchange (Bitcoin: `inv`, `getdata`, `tx`; Ethereum: similar hash announcements)
- the hash identifies the transaction, and filters duplicates: a `received` set again



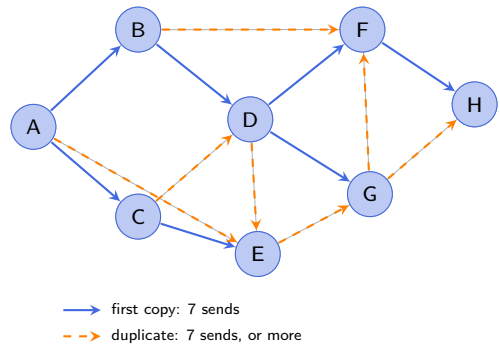
CometBFT's mempool: flooding

- a transaction from a client or a peer is checked: dropped if invalid, or already in a cache of the latest 10 000
- a valid one is appended to the mempool, in arrival order



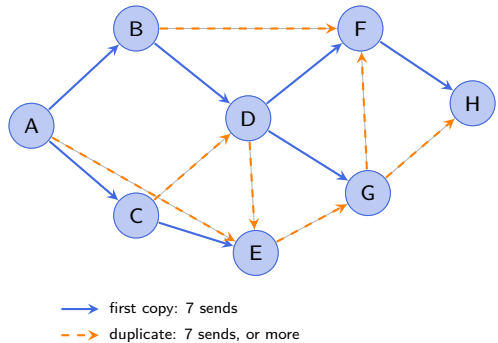
CometBFT's mempool: flooding

- a transaction from a client or a peer is checked: dropped if invalid, or already in a cache of the latest 10 000
- a valid one is appended to the mempool, in arrival order
- one routine per peer sends it each transaction **in full**, unless the peer sent it; it waits while the peer lags more than a block behind



CometBFT's mempool: flooding

- a transaction from a client or a peer is checked: dropped if invalid, or already in a cache of the latest 10 000
- a valid one is appended to the mempool, in arrival order
- one routine per peer sends it each transaction **in full**, unless the peer sent it; it waits while the peer lags more than a block behind
- push only, to *every* peer: fast, hard to censor, but every link carries every transaction: **duplicates**



practical 2: build this mempool

Gossip in blockchains: other approaches

Bitcoin	transactions: hash first, body on request; blocks: *compact*, only the short IDs of their transactions
Ethereum	execution layer: transaction hash announcements over devp2p; consensus layer: blocks and attestations over libp2p *GossipSub*
Solana	no public mempool: transactions are forwarded to the upcoming leaders; blocks propagate down a tree of validators (Turbine)
Avalanche	gossip *is* the consensus: nodes repeatedly sample k random peers and adopt the majority preference
Sui, Aptos	*Narwhal*-style DAG mempools (Aptos: Quorum Store) separate dissemination from ordering

- **no global view**: each node has its own mempool; two nodes may disagree on what is pending
- **delay**: gossip takes a number of rounds, logarithmic in the network size, times the network latency
- **censorship**: gossip gives no guarantee that a transaction reaches the miner
- **front-running**: pending transactions are public. Observers can insert their own transaction ahead of yours by paying a higher fee (MEV, lecture 8)
- **blocks too**: blocks are disseminated the same way; to save bandwidth, relays send only identifiers of transactions the peer likely holds already

- practical 2: **the waiting room of a blockchain**
- implement a mempool in the style of CometBFT: deduplication, validity (balance and nonce), first come first served selection, recheck after a block, then flooding between peers over TCP, with Protocol Buffers on the wire
- in Go: `practicals/mempool`

1. Motivation
2. The mempool: pending and valid transactions
3. Distributed computing: model and correctness
4. Reliable broadcast
5. Gossip-based protocols
6. Wrap-up

Takeaways

- the **mempool** is the node's buffer of pending transactions: gossip to disseminate, fee ordering (or arrival order) to select
- a node keeps and relays only **valid** transactions: the sender can pay, and the **nonce** is not stale, which prevents replays and orders the transactions of a sender
- specifying dissemination means choosing a **model** (links, failures, synchrony, graph) and **properties**: **safety** (nothing bad ever happens) and **liveness** (something good eventually happens)
- **reliable broadcast**: relay before delivering; correct, but about n^2 messages and a complete graph
- at scale, **gossip**: random partners, logarithmic spreading time, robustness
- **anti-entropy** is reliable but slow and chatty; **rumor mongering** is fast but probabilistic; combine them
- **CometBFT** floods transactions to every peer: fast, hard to censor, but many duplicates

- **practical 2**: the memory pool
- **lecture 3**: consensus, FLP, and Paxos — agreeing on *which* transactions, and in which order

Readings

- Alpern and Schneider, *Recognizing safety and liveness* [†]
- Demers et al., *Epidemic algorithms for replicated database maintenance* [‡]
- Doerr and Kostrygin, *Randomized rumor spreading revisited* [§]
- Danezis et al., *Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus*, which specifies a mempool (Narwhal) [¶]

[†] B. Alpern et al. "Recognizing Safety and Liveness". *Distributed Computing* (1987).

[‡] A. Demers et al. "Epidemic Algorithms for Replicated Database Maintenance". *PODC*. 1987.

[§] B. Doerr et al. "Randomized Rumor Spreading Revisited". *ICALP*. 2017.

[¶] G. Danezis et al. "Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus". *EuroSys*. 2022.